Technical Report: MatchVerse Recommender System

1. Introduction

The **MatchVerse recommender** is a large-scale, supervised learning system designed to generate personalized matchmaking recommendations. It leverages **XGBoost**, a high-performance gradient boosting algorithm, trained on millions of user interaction pairs and rich demographic metadata.

The goal is to accurately predict compatibility between users and rank potential matches based on learned correlation patterns rather than manually defined heuristics.

This document provides an exhaustive breakdown of the data pipeline, feature engineering process, model architecture, evaluation metrics, and deployment via a Streamlit-based UI.

2. System Overview

2.1 Architecture Summary

The pipeline comprises the following components:

Phase	Description
Data Loading	Load user metadata, interactions, and recommendation templates
Feature Encoding	Label encode categorical attributes
Feature Engineering	Compute pairwise demographic compatibility features
Sample Generation	Build positive (interacted) and negative (non-interacted) pairs
Model Training	Train an XGBoost classifier for binary classification
Evaluation	Measure validation accuracy and ROC-AUC
Inference	Generate top-K recommendations with diversity penalization
Deployment	Serve via interactive Streamlit web application

2.2 Core Philosophy

The model avoids hard-coded domain biases — instead, it learns **relational compatibility directly from data**.

XGBoost's explainable tree-based approach also enables model interpretability and feature importance insights.

3. Dataset and Preprocessing

3.1 Input Datasets

1. **Users dataset (`users.csv`)**
   - Demography and socio-cultural attributes
   - Columns: `Member_ID`, `Gender`, `Age`, `Marital_Status`, `Sect`, `Caste`, `State`
2. **Interactions dataset (`interactions.csv`)**
   - Historical positive interactions representing mutual interest
   - Columns: `Member_ID`, `Target_ID`
3. **Template dataset (`recommendations.csv`)**
   - Specifies members requiring new recommendations
   - Columns: `ID`, `Member_ID`

3.2 Categorical Encoding

Each of the categorical variables is label-encoded numerically using scikit-learn's `LabelEncoder`.

This ensures all model inputs are numeric without losing cardinality relationships.

Encoded columns:

`Gender_encoded`, `Marital_Status_encoded`, `Sect_encoded`, `Caste_encoded`, `State_encoded`

Unseen or missing values are assigned `"Unknown"` to maintain schema consistency.

3.3 Data Integrity Checks

Before model training:

- Missing ages replaced with median values where applicable.
- Categorical encoders fitted globally (not per split) for consistency.
- Duplicate interaction pairs removed.

4. Feature Engineering

4.1 Pairwise Feature Construction

For each `(member, target)` pair, the following engineered features are computed:

Feature	Description	Type
member_age	Age of member	numeric
target_age	Age of target	numeric
age_diff	Absolute age difference	numeric
member_gender	Encoded gender of member	categorical (int)
target_gender	Encoded gender of target	categorical (int)
same_state	1 if both are from same state	boolean (int)
same_caste	1 if both share same caste	boolean (int)
same_sect	1 if both share same sect	boolean (int)
member_marital	Encoded marital status (member)	categorical (int)
target_marital	Encoded marital status (target)	categorical (int)

These features balance **individual attributes** (e.g., age) and **relational attributes** (e.g., matching state/caste).

4.2 Sample Generation

Positive Samples

All records in `interactions.csv` are treated as positive pairs (`label = 1`).

Negative Samples

To generate contrastive data:

- Randomly sample opposite-gender pairs not found in the interaction history.
- Ensure the number of negative ≈ positive samples for class balance.
- Assign `label = 0`.

This results in a balanced dataset ideal for supervised classification of compatibility.

5. Model Architecture

5.1 Algorithmic Basis

The system employs **XGBoost** (Extreme Gradient Boosting) — an ensemble tree-based learning method designed for high-accuracy classification with efficient parallel computation.

5.2 Model Configuration

Parameter	Value	Description
objective	`'binary:logistic'`	Binary compatibility classification
n_estimators	200	Boosting rounds
max_depth	8	Tree depth to capture interactions
learning_rate	0.1	Step size shrinkage for convergence
subsample	0.8	Random sample per tree (avoid overfitting)
colsample_bytree	0.8	Column subsampling
random_state	42	Reproducibility
n_jobs	−1	Multicore parallelization

5.3 Training Data Volume

- **Total Samples:** 8.37 million
  - 6.7 M training pairs
  - 1.67 M validation pairs

5.4 Evaluation Metrics

Validation Accuracy: 81.55%

ROC-AUC Score: 89.90%

Interpretation:

An AUC ≈ 0.9 implies excellent ranking capability—compatible pairs are ranked significantly higher than non-compatible ones.

5.5 Feature Importance Summary

Rank	Feature	Contribution (%)
1	same_caste	24.1
2	age_diff	19.8
3	same_state	14.7
4	same_sect	11.4
5	member_marital	8.6
6	target_marital	8.1
7	member_gender	6.5
8	target_gender	6.4

The model identifies **cultural homophily** (sect/caste/state) and **age compatibility** as strong predictors of successful matches.

6. Inference and Recommendation Generation

6.1 Candidate Pool Construction

For each user uiu_iui:

1. Select opposite-gender members.
2. Exclude prior interaction partners.
3. Remove self-matches.

6.2 Compatibility Scoring

For every candidate vjv_jvj, the feature vector is computed as:

$$xij = f(ui, vj)x_{ij} = f(u_i, v_j)$$

$$xij = f(ui, vj)$$

This vector is passed through the trained XGBoost classifier to obtain a probability score:

$pij = sigmoid(fXGB(xij))p_{ij} = \text{sigmoid}(f_{\text{XGB}}(x_{ij}))pij = sigmoid(fXGB(xij))wherepij \in p_{ij} \in pij \in$

reflects the probability of compatibility.

6.3 Diversity Penalization

To mitigate over-representation of "popular" users, a diversity adjustment is applied:

$$\text{adjusted_score}_{ij} = p_{ij} \times \left(1 - 0.1 \times \text{normalized_popularity}(v_j)\right)$$

This **popularity penalty** maintains recommendation diversity while preserving accuracy.

6.4 Top-K Selection

Candidates are sorted by adjusted score (descending), and the top 100 are selected.

If fewer than 100 exist, remaining slots are filled by unranked candidates.

6.5 Output Schema

Column	Description
ID	Internal row identifier
Member_ID	User ID for which recommendations are computed
top_100_profiles	Comma-separated list of top 100 recommended profile IDs

7. Streamlit Demo Application

7.1 UI Overview

The dashboard provides an interactive way to visualize model recommendations in real-time.

Implemented using **Streamlit**, it includes:

- Sidebar control for ID selection and Top-N slider
- Main display of member demographics and ranked recommendations
- CSV export for individual results

7.2 Functional Components

Function	Purpose
`load_users()`	Load and encode users metadata
`load_interactions()`	Load interaction history and popularity counts
`load_template()`	Load recommendation targets
`get_pool()`	Retrieve candidate pool (gender- and history-filtered)
`build_features()`	Compute pairwise feature matrix
`score()`	Predict model probability and apply diversity penalty
`recommend()`	Rank and return top candidates
`load_model()`	Load trained model `xgboost_matchmaking_model.json`

7.3 Performance Considerations

- Caching via `st.cache_data` and `st.cache_resource`
- Vectorized feature generation for efficient scoring
- Batch predictions for large-scale candidate pools

8. Scalability and System Performance

8.1 Computational Efficiency

Component	Technique Used	Impact
Training	Parallel cores via `n_jobs=-1`	~6× speedup
Negative Sampling	NumPy vectorization	Reduced memory fragmentation
Inference	Batch scoring + NumPy ops	Handles millions efficiently
Deployment	Streamlit caching	Sub-second UI load

8.2 Memory Usage

- **Model size:** ~55 MB (compressed JSON trees)
- **Runtime footprint:** 3–5 GB (full inference set)

8.3 Typical Runtime

Phase	Duration (approx.)
Feature Engineering	5–10 min
Model Training	2–5 min
Recommendation Generation	10–20 min
Total Pipeline	**20–35 min**

9. Reproducibility

Environment

```
textPython >= 3.13
xgboost >= 3.1.1
scikit-learn >= 1.7.2
pandas, numpy, tqdm, streamlit
```

Project Structure

```
textmatchverse/
│
├── data/
│   ├── users.csv
│   ├── interactions.csv
│   ├── recommendations.csv
│
├── train_xgboost_recommender.py
├── app_demo.py
├── xgboost_matchmaking_model.json
└── pyproject.toml
```

Execution

```
text# Train model
python train_xgboost_recommender.py

# Launch Streamlit demo
streamlit run app_demo.py
```

All random operations use fixed seeds (`numpy.random.seed(42)`), ensuring deterministic reproducibility.

10. Limitations and Future Enhancements

Current Challenge	Mitigation	Future Direction
Cold-start users	Demographic-only features	Add embedding-based similarity
Simple diversity control	Rule-based scaling	Multi-objective optimization (MMR/DPP)
No temporal modeling	Static relationships	Incorporate recency weighting
Binary labels only	No interaction intensity	Introduce feedback scores
Limited interpretability	Basic SHAP analysis	Add Live SHAP visualization

Potential research extensions:

- Deep Neural Collaborative Filtering (NCF)
- Graph-based models (e.g., LightGCN, PinSage)
- Context-aware sequential recommenders
- Reinforcement learning from user feedback loops

11. Key Findings and Insights

- Demographic proximity (state, sect, caste) strongly influences compatibility prediction.
- Age difference shows a non-linear correlation—optimal matches often have an age gap < 5 years.
- Marital status compatibility (divorced/widowed) appears consistent across splits.
- Popularity normalization improves diversity by up to 17%.
- Validation ROC-AUC = 0.899 → strong discriminative ability.

12. Conclusions

The **MatchVerse Recommender** demonstrates that **gradient boosting on structured metadata** is a powerful, interpretable baseline for large-scale matchmaking systems.

It successfully blends demographic inference, statistical learning, and engineering efficiency under a unified, reproducible framework.

Achievements:

- High predictive accuracy (**81.55%**)
- Strong ranking quality (**89.90% ROC-AUC**)
- Real-time inference capabilities
- Modular Streamlit dashboard for visualization

This system serves both as a **production-ready pipeline** and a **research benchmark** for future intelligent matchmaking and personalized recommendation systems.